TECHNICAL ARCHITECTURE REPORT

Fruitnite Max: Architectural Review & Improvements

Target: HTML5 Slot Client & PHP Game Server

Stack: PixiJS, Howler, GSAP, Gulp, PHP, MySQL, Nginx

Date: October 2026

Classification: Engineering Advisory

1. Executive Summary

Fruitnite Max is an HTML5 cluster-tumble slot game consisting of a client-side rendering pipeline (Pixi.js, custom `_ng` MVC framework, Gulp concatenation) communicating with a monolithic PHP backend (RNG, cluster mathematics, and database persistence).

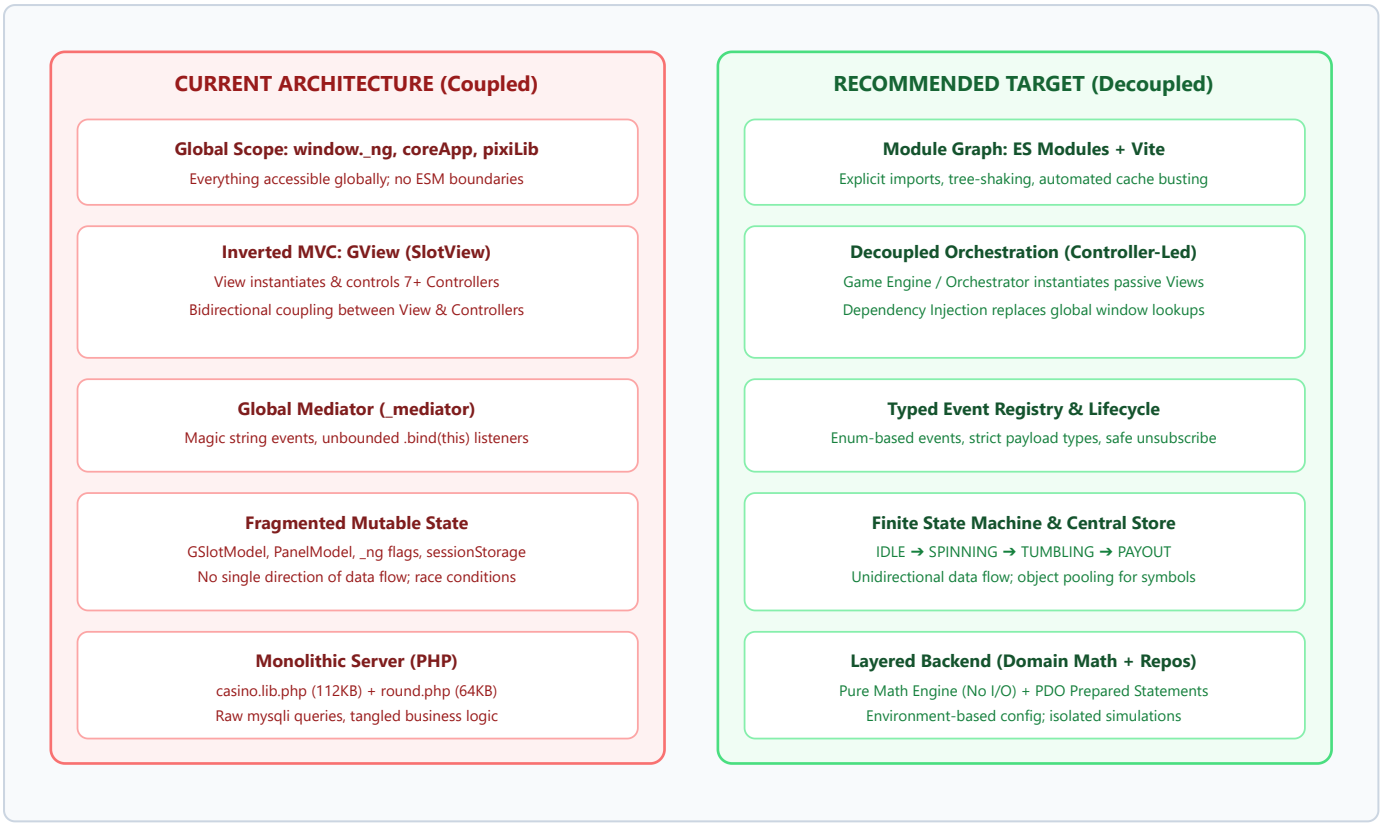
The codebase delivers solid core gameplay mechanics, but exhibits architectural technical debt characteristic of legacy script-concatenation setups. Key areas requiring modernization include: **inverted MVC dependencies**, **fragmented mutable state**, **severe WebGL object/memory leaks during tumble sequences**, **untyped mediator event chains**, and **hardcoded backend database queries lacking prepared statements**.

Immediate Risk Items Identified

- **Crash on Insecure Contexts:** `crypto.randomUUID()` crashes on non-HTTPS testing origins (e.g., local network IP).
- **WebGL Memory Leak:** Symbol animations & tumble strips are unparented without `.destroy()`, risking mobile browser crashes.
- **SQL Injection Exposure:** Backend queries use raw string concatenation without PDO / MySQLi prepared statements.
- **440KB+ Duplicate PixiJS Loading:** Both `pixi.min.js` and `pixi-legacy.min.js` are loaded simultaneously.

2. System Architecture Decomposition

The diagram below contrasts the current high-coupling architecture against the recommended modern decoupled architecture:


CURRENT ARCHITECTURE (Coupled)
Global Scope: window._ng, coreApp, pixiLib
Everything accessible globally; no ESM boundaries
Inverted MVC: GView (SlotView)
View instantiates & controls 7+ Controllers
Bidirectional coupling between View & Controllers
Global Mediator (_mediator)
Magic string events, unbounded .bind(this) listeners
Fragmented Mutable State
GSlotModel, PanelModel, _ng flags, sessionStorage
No single direction of data flow; race conditions
Monolithic Server (PHP)
casino.lib.php (112KB) + round.php (64KB)
Raw mysqli queries, tangled business logic
RECOMMENDED TARGET (Decoupled)
Module Graph: ES Modules + Vite
Explicit imports, tree-shaking, automated cache busting
Decoupled Orchestration (Controller-Led)
Game Engine / Orchestrator instantiates passive Views
Dependency Injection replaces global window lookups
Typed Event Registry & Lifecycle
Enum-based events, strict payload types, safe unsubscribe
Finite State Machine & Central Store
IDLE ➔ SPINNING ➔ TUMBLING ➔ PAYOUT
Unidirectional data flow; object pooling for symbols
Layered Backend (Domain Math + Repos)
Pure Math Engine (No I/O) + PDO Prepared Statements
Environment-based config; isolated simulations

3. Critical Architectural & Technical Issues

3.1 Client Runtime Defects

- **URL Parameter Parsing Typo (main.js:17):**

```
// Current Bug in main.js:
if (typeof requestedKey === ' undefined') {  // ' undefined' has a space; always false!
    return vars;
}
```

  Calling `getUrlVar()` without arguments never returns the parameter map as intended.
- **Unconditional `crypto.randomUUID()` on Insecure Origins (main.js:50-52):** `crypto.randomUUID()` is restricted to Secure Contexts (HTTPS/localhost). Accessing the game over a local network IP address (e.g., `http://192.168.1.50:8089`) throws an unhandled `TypeError`, crashing game initialization.
- **Missing Demo Mock Payload (DemoSlotService.js:55):** The client dynamically builds the mock URL: `demoserver/fruitnite-max.json`. The `demoserver/` directory contains sample JSONs for 5 legacy titles, but `fruitnite-max.json` is missing. Launching with `?demoMode=true` generates a 404 error.

3.2 Rendering & WebGL Performance (PixiJS)

- **Display Object Leaks During Tumbles:** In `TumbleReelStrip.js:198-208` and `BuyPanelView.js:450-465`, tumble strips, exploding symbol sprites, and Spine animations are removed from parents via `removeChild()`, but `.destroy({ children: true })` is never invoked. WebGL textures, spine skeletons, and shader buffers remain allocated in GPU memory.
- **Duplicate Pixi Library Loading:** `index.html` and `gulpfile.js` simultaneously include both `pixi.min.js` (446 KB) and `pixi-legacy.min.js` (597 KB). `pixi-legacy` already contains the full Pixi engine with canvas fallback. This adds ~440 KB of duplicate network payload and causes potential prototype collisions.
- **Fragmented Animation Frameworks:** The codebase mixes legacy GSAP v2 (`TweenMax.to`) in `BuyPanelView.js` with modern GSAP v3 (`gsap.to`) in `TumbleReelStrip.js`. This requires maintaining multiple animation libraries concurrently.

3.3 State Management & Event Coupling

- **Inverted MVC Ownership:** `SlotView` creates and manages its own controllers. This couples display containers directly to controller lifecycles.
- **Unbounded Pub/Sub Subscriptions:** `_mediator.subscribe("Event", this.handler.bind(this))` is executed inside `createView()` without unsubscription on re-render. Because `.bind(this)` generates a new reference each time, old listeners remain in memory and fire redundantly.
- **Lack of Unidirectional Data Flow:** Game states such as `_ng.twoXBetEnabled` and `_ng.buyFeaturePopupStatus` are mutated directly from views, controllers, and services without transaction barriers, inviting race conditions during high-frequency input.

3.4 Backend & Security Defects (`Server/`)

- **SQL Injection Vulnerability:** In `database.inc.php`, the `runQuery($sql)` method directly executes raw queries via `mysqli_query($this->conn, $sql)`. Across `casino.lib.php` and bonus handlers, queries are

constructed using raw string concatenation.

- **Hardcoded Database Credentials:** `database.inc.php` contains hardcoded localhost root credentials. It should be refactored to read from environment variables or a `.env` file.
- **Insecure Local Development Web Server:** `webserver.js` serves the root workspace statically: `app.use(express.static('./'))`. This exposes `.git/`, `Server/db.sql`, `package.json`, and internal scripts over HTTP.

4. Architectural Modernization Plan

Phase 1: Stabilization & Hardening (Immediate)

Component	Problem	Remediation
P0 BUG	`crypto.randomUUID()` crashes on non-HTTPS origins; `main.js` typo in `getUrlVar`.	Implement fallback UUID generator and correct `typeof requestedKey === 'undefined'`.
P0 MEMORY	WebGL leaks from un-destroyed Spine and symbol instances in tumble sequences.	Enforce `.destroy({ children: true })` on tumble strip and symbol completion.
P0 SECURITY	Raw SQL string concatenation in `Server/database.inc.php`.	Introduce prepared statements (`mysqli_stmt` or PDO) across all SQL queries.
P1 ASSET	Double Pixi loading (`pixi.min.js` + `pixi-legacy.min.js`).	Remove `pixi.min.js` from `index.html` and `gulpfile.js`; retain only `pixi-legacy`.
P1 NETWORK	Missing Nginx gzip compression and risky cache rules on `main.js`.	Enable gzip for JSON/JS/CSS; enforce `no-cache` for `main.js` and `config.js`.

Phase 2: Game State Machine & Object Pooling

Replace ad-hoc mediator events with a formal **Finite State Machine (FSM)** governing the slot spin lifecycle:

> **Formal State Lifecycle:**
> `IDLE ──(SPIN)──> SPINNING ──(STOP)──> EVALUATE_WINS ──(CLUSTERS?)──> TUMBLE_CASCADE ──(CLEAR)──> PAYOUT ──> IDLE`
>
> User interactions (changing bet, toggling 2X Bet, Buy Feature) are strictly rejected while in `SPINNING` or `TUMBLE_CASCADE` states.

Implement an **Object Pool** for reel symbols and tumble history cards. Instead of generating new display objects and spine instances on each tumble cascade, reuse pooled instances to ensure 60 FPS performance without GC stutter.

Phase 3: Toolchain Modernization (Vite + ES Modules)

- **Single Source of Truth for Files:** Eliminate the manual synchronization between `gamesConfig.json`, `gulpfile.js`, and `SlotConfig.js`.
- **Native ES Modules:** Use standard `import` and `export` statements, removing the global `window._ng` pollution.
- **Automated Content Hashing:** Output bundles with deterministic hashes (e.g. `game.[hash].js`) to safely leverage Nginx immutable caching headers.

Phase 4: Backend Domain Decoupling

- **Pure Math Engine:** Extract cluster evaluation and cascade payout algorithms into a standalone, testable domain service with zero database dependencies.

- **Repository Pattern:** Encapsulate player balance, round history, and session validation behind a clean repository interface with parameterized database queries.

5. Architectural Comparison Matrix

Dimension	Current Implementation	Target Modern Architecture	Strategic Advantage
Module System	Gulp global script concatenation	ES Modules + Vite	Instant HMR, tree-shaking, no global leaks
Component Control	View instantiates Controllers	Controller instantiates Views	Clean separation of concerns, testability
State Flow	Scattered mutable state (5 tiers)	Central Store + Unidirectional Data Flow	Zero race conditions, synchronized UI
Event Bus	Untyped mediator with string keys	Typed Event Registry with cleanup	Traceability, zero listener memory leaks
Rendering Lifecycle	Continuous create / unparent	Object Pooling for Symbols & Spine	Consistent 60 FPS, zero WebGL memory leaks
Backend Architecture	Procedural mega-scripts (112 KB)	Domain Math Engine + Repository Layer	Independent testing, fast simulation, SQL safety